

# 네트워크 통신 정리

## 1. 고정ip설정 방법

ip고정해야 랜선 직접 연결상태에서 DHCP없이 서로 같은 네트워크 대역 맞춰져서 통신 가능하다고 하네요.

### 1. 라즈베리파이

```
sudo nano /etc/dhcpd.conf
```

```
interface eth0
static ip_address=192.168.10.2/24
```

- RTSP사용시 주소 : rtsp://192.168.10.2:8554/stream

## 2. PC

### 1. 제어판->네트워크 공유센터

네트워크 및 공유 센터

← → ⌵ ↑ > 제어판 > 네트워크 및 인터넷 > 네트워크 및 공유 센터

제어판 홈

어댑터 설정 변경

고급 공유 설정 변경

미디어 스트리밍 옵션

#### 기본 네트워크 정보 보기 및 연결 설정

활성 네트워크 보기

**iptime5G 2**  
공용 네트워크

액세스 형식: 인터넷  
연결:  Wi-Fi(iptime5G)

네트워크 설정 변경



새 연결 또는 네트워크 설정

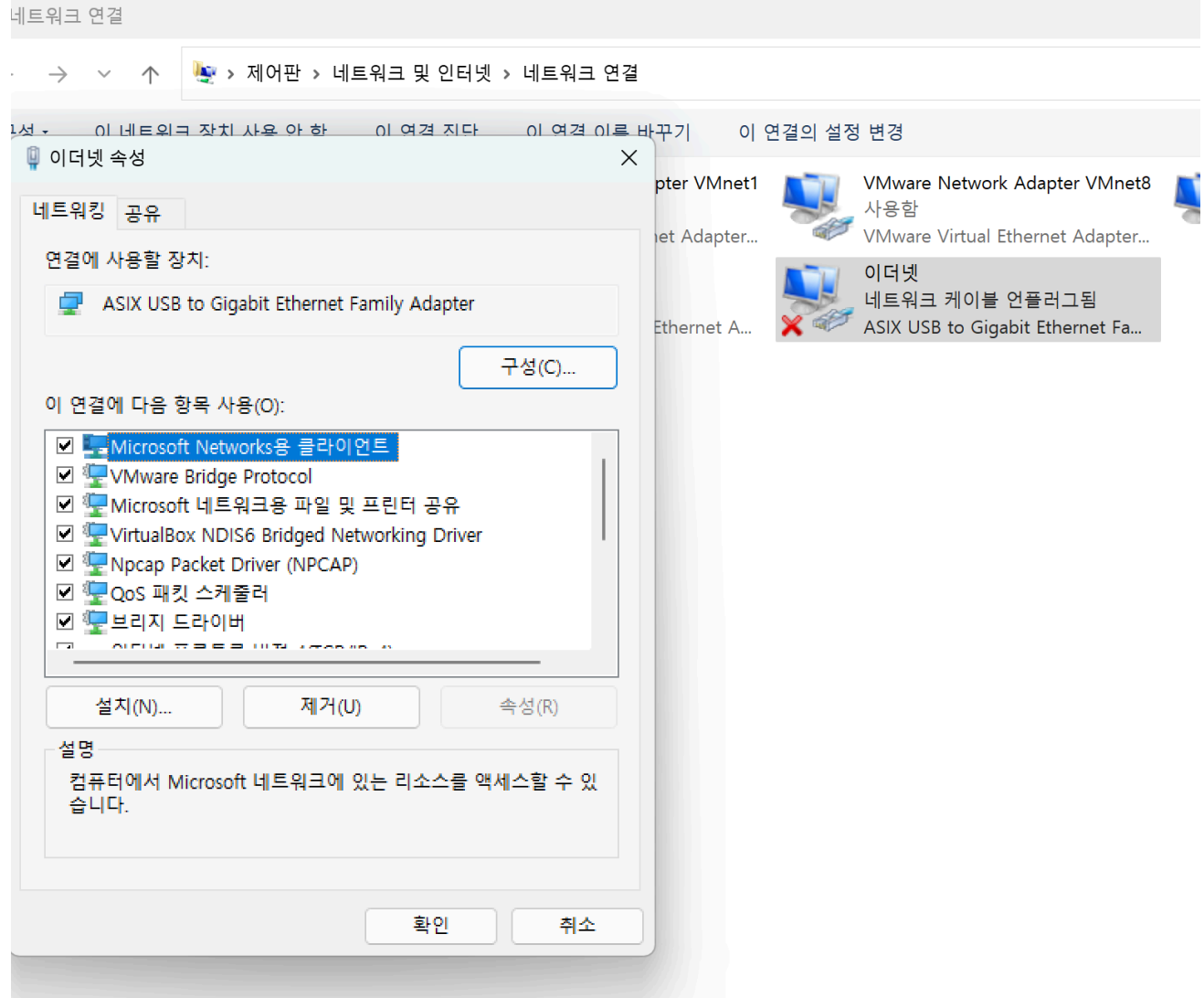
광대역, 전화 접속 또는 VPN 연결을 설정하거나 라우터 또는 액세스 지점을 설정합니다.



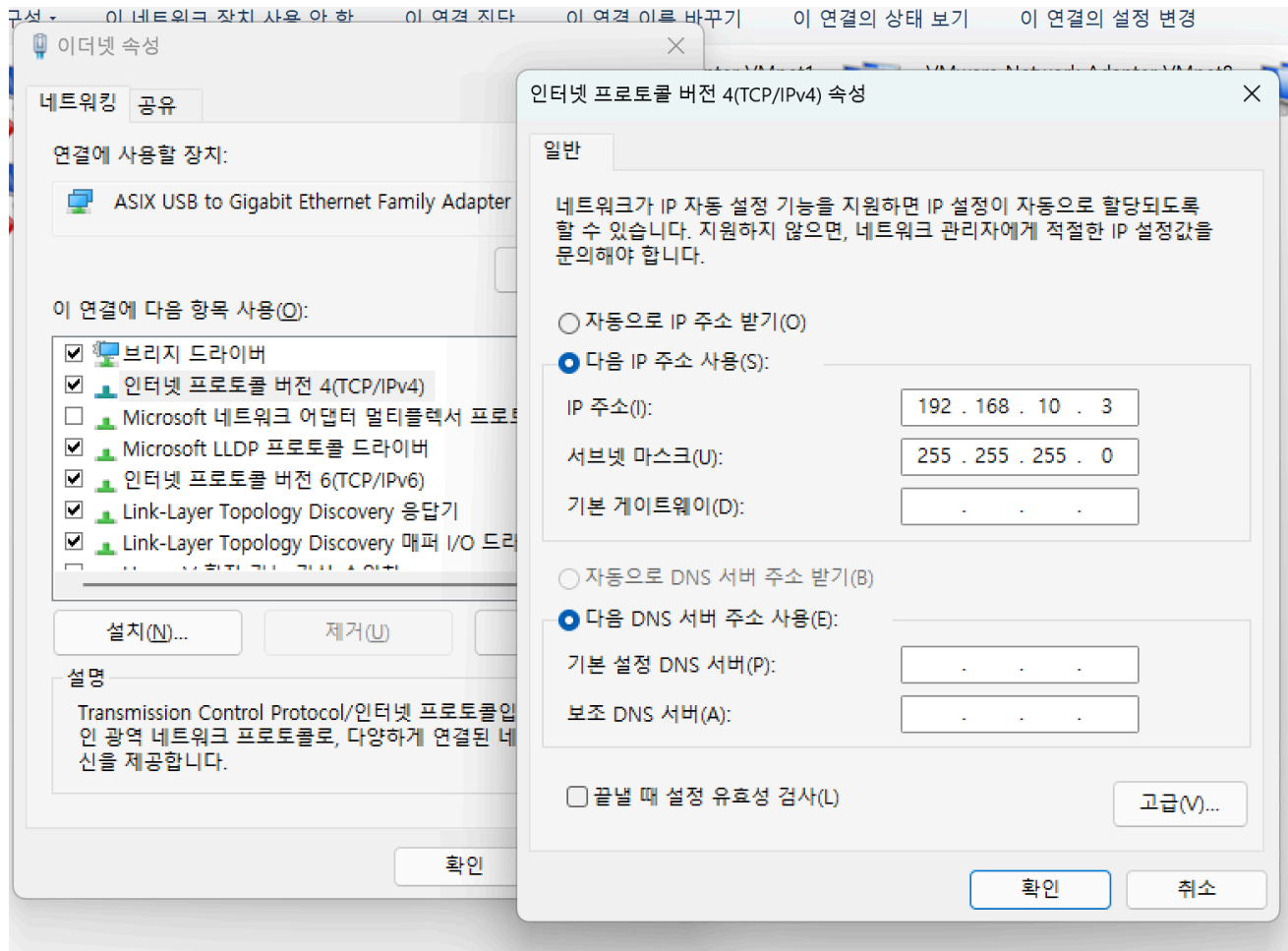
문제 해결

네트워크 문제를 진단 및 해결하거나 문제 해결 정보를 얻습니다.

## 2. 어댑터 설정 변경->이더넷 선택



### 3. IPv4선택후 다음ip주소 사용으로 바꿔서 ip, 서브넷 고정



## 2. 서버 코드

```
import gi

gi.require_version("Gst", "1.0")
gi.require_version("GstRtspServer", "1.0")

from gi.repository import Gst, GstRtspServer, GLib

class CameraRTSPServer:
    def __init__(self):
        Gst.init(None)

        self.server = GstRtspServer.RTSPServer()
        self.server.set_service("8554")

        factory = GstRtspServer.RTSPMediaFactory()
        factory.set_shared(True)
```

```

    pipeline = (
        "( "
        "v4l2src device=/dev/video0 "
        "! video/x-raw,width=640,height=480,framerate=30/1 "
        "! videoconvert "
        "! x264enc tune=zerolatency speed-preset=ultrafast bitrate=1000 "
        "! rtph264pay config-interval=1 name=pay0 pt=96 "
        ")"
    )

    factory.set_launch(pipeline)

    mounts = self.server.get_mount_points()
    mounts.add_factory("/stream", factory)

    self.server.attach(None)

    print("RTSP 서버 시작")
    print("접속 주소: rtsp://라즈베리파이IP:8554/stream")

if __name__ == "__main__":
    server = CameraRTSPServer()

    loop = GLib.MainLoop()
    loop.run()

```

## 서버 코드 핵심 요약

```

Gst.init(None)

self.server = GstRtspServer.RTSPServer()
self.server.set_service("8554")

factory = GstRtspServer.RTSPMediaFactory()
factory.set_shared(True)

```

- GStreamer 초기화
- RTSP 서버 생성 및 포트 설정 (8554)
- 스트림 생성 객체(factory) 준비

```

v4l2src device=/dev/video0
! video/x-raw,width=640,height=480,framerate=30/1
! videoconvert

```

```
! x264enc tune=zerolatency speed=ultrafast bitrate=1000
! rtph264pay name=pay0 pt=96
```

- 카메라 입력
- 해상도/FPS 설정
- 포맷 변환
- H.264 압축
- RTP 패킷화

```
factory.set_launch(pipeline)

mounts = self.server.get_mount_points()
mounts.add_factory("/stream", factory)

self.server.attach(None)
```

- 파이프라인을 RTSP 서버에 등록
- /stream 경로로 스트림 생성
- 서버 실행

### 3. 클라이언트 코드

```
import cv2

RTSP_URL = "rtsp://192.168.10.2:8554/stream"

cap = cv2.VideoCapture(RTSP_URL)

if not cap.isOpened():
    print("RTSP 연결 실패")
    exit()

print("RTSP 연결 성공")

while True:
    ret, frame = cap.read()

    if not ret:
        print("프레임 수신 실패")
        break

    cv2.imshow("RTSP Client", frame)
```

```
# q 누르면 종료
if cv2.waitKey(1) & 0xFF == ord("q"):
    break

cap.release()
cv2.destroyAllWindows()
```

## 서버 코드 핵심 요약

```
cap = cv2.VideoCapture(RTSP_URL)
```

- RTSP 주소 접속
- RTP 패킷 수신
- H.264 압축 데이터 추출
- 디코딩(FFmpeg로 한다고 하네요.)
- OpenCV가 다룰 수 있는 frame 이미지로 변환

```
ret, frame = cap.read()
```

- ret: 성공 여부
- frame: 이미지 데이터

```
cv2.imshow("RTSP Client", frame)
```

- 수신한 프레임을 화면에 출력
- 반복 실행 → 실시간 영상처럼 보임

## 4. GStreamer 사용 이유

아래에 있는거 애가 대신해준다고 하네요.

카메라 캡처

- 인코딩(H.264)
- 프레임 분할
- RTP 패킷 생성
- 소켓 전송
- RTSP 제어 구현
- 버퍼/지연 처리

GStreamer는 카메라 입력부터 인코딩, 패킷화, 네트워크 전송까지의 전체 스트리밍 과정을 자동으로 처리해주기 때문에, 복잡한 소켓 통신과 영상 처리 코드를 직접 작성할 필요가 없다.

## 5. 동작 확인



- 약 0.5초정도 지연이 있습니다. 정한님이 라즈베리파이 안쓰고 로컬로만 해봤는데도 지연 생기는거로 보서는 카메라나 네트워크문제보단 그냥 rtsp자체가 약간 느린거 같기도 해요.
- 아직 랜선 뽑히고 재연결시 재전송하는건 어디 버퍼에 막히는건지 코드가 이상해서 기능 추가 없이 가장 기본적인 코드만 올릴게요.